SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I **M Sivam Kumar** **(192521017)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

M Sivam Kumar
Signature of the student:

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Q1. ATM Withdrawal Module – Reverse Engineering of Black-box Testing

Reverse engineering the ATM withdrawal module means identifying its hidden validation rules and test design strategy by observing inputs, outputs, and error messages without accessing the source code.

1. Functional Decomposition: Divide the module into card/PIN validation, account balance verification, withdrawal limit checking, cash availability, transaction processing, and receipt generation.

2. Equivalence Class Partitioning (ECP): Assuming the permitted withdrawal range is ₹500–₹25,000, divide amounts into below-minimum, valid, and above-maximum classes. Representative values could be ₹200, ₹5,000, and ₹30,000. Separate classes should also cover insufficient balance and invalid PINs.

3. Boundary Value Analysis (BVA): Test ₹499, ₹500, ₹501, ₹24,999, ₹25,000, and ₹25,001. These values help identify incorrect limit comparisons and rounding errors.

4. Adequacy and Reconstruction: If existing logs show only successful withdrawals and common invalid inputs, the original strategy may have focused on normal cases. Add tests for insufficient funds, unavailable cash, daily limits, network failure, and repeated withdrawal requests.

Conclusion: Observed outputs can reveal likely validation rules, but they cannot prove the exact internal implementation. Combining ECP, BVA, and transaction-failure testing creates a more reliable test suite and reduces the risk of incorrect debits.

Q2. Software Module – Reverse Engineering White-box Testing

When source code snippets and execution traces are available but test documentation is missing, the testing strategy can be reconstructed by mapping executed statements and decisions into a control flow graph.

1. CFG Construction: Represent each statement block as a node and each possible transfer of control as an edge. For a module containing input validation, a decision, a loop, and a final output, the graph should include both decision outcomes, the loop-back edge, and the exit path.

2. Decision and Path Identification: Identify every conditional branch and loop. For example, a validation condition may lead either to processing or to an error message. A loop may execute zero times, once, or several times.

3. Coverage Evaluation: Statement coverage checks whether statements execute, branch coverage checks both outcomes of decisions, and path coverage examines sequences of execution. Existing traces can show which paths were exercised and which remain unverified.

4. Reconstructed Test Strategy: Create test cases for valid input, invalid input, each decision outcome, loop boundary conditions, and error handling. Calculate cyclomatic complexity using $V(G) = E - N + 2$ to estimate the number of independent paths.

Conclusion: A CFG-based reconstruction provides a structured explanation of the existing tests

and exposes missing execution routes. Complete path coverage may not be practical for complex loops, so independent paths and high-risk combinations should be prioritized.

Q3. Web Login and Transaction System – Reverse Engineering Testing Techniques

Defect logs and test results can reveal weaknesses in a web application's testing approach. For example, repeated login failures may indicate inadequate authentication testing, while duplicate transactions may suggest missing retry and transaction-state checks.

1. Functional Decomposition: Divide the application into login validation, session management, transaction authorization, balance verification, transaction execution, and confirmation.

2. Infer Black-box Techniques: Defects around minimum and maximum transaction amounts may indicate inadequate BVA. Errors involving invalid credentials or account details may indicate incomplete ECP. Defects occurring only with particular combinations suggest insufficient decision table or interaction testing.

3. Infer White-box Techniques: Repeated failures in authorization or transaction-processing paths may indicate missing branch or path coverage. However, defect logs alone cannot establish which white-box technique was actually used.

4. Reconstructed Strategy: Test valid and invalid credentials, expired sessions, transaction amounts at boundaries, insufficient balances, and repeated requests. Apply ECP and BVA to inputs, then use CFG-based path testing for authorization, execution, and error handling.

Conclusion: Combining defect-pattern analysis with black-box and white-box testing helps identify probable coverage gaps. The reconstructed strategy addresses both incorrect inputs and internal execution paths, improving login security and transaction reliability.

Q4. Legacy Inventory Management – Reverse Engineering from Transaction Logs

Successful and failed inventory logs can help reveal hidden stock-update rules, low-stock thresholds, and validation conditions. The objective is to infer the business logic from observed outcomes and then design tests that challenge those inferred rules.

1. Functional Decomposition: Divide the system into stock input validation, quantity update, stock-level calculation, low-stock alert generation, and transaction logging.

2. Identify Equivalence Classes: For a stock quantity, consider negative values, zero, valid positive quantities, and values exceeding the permitted storage capacity. For the low-stock alert, consider quantities below, at, and above the suspected threshold.

3. Boundary Testing: If the logs suggest that an alert is generated at five units or fewer, test quantities of 4, 5, and 6 units. This is a hypothesis based on the observed behavior and must be verified against the actual system rules.

4. Reconstructed Test Design: Test stock addition, stock reduction, zero-stock conditions, invalid quantities, repeated updates, and concurrent updates. Verify that alerts appear at the correct threshold and that failed transactions do not corrupt inventory records.

Conclusion: Log-based reverse engineering can identify probable boundaries and missing

cases, but observed patterns should be confirmed before being treated as official requirements. ECP and BVA provide a repeatable way to validate the inferred inventory logic.

Q5. Library Management Module – Reverse Engineering from CPU and Memory Profiles

A compiled executable and its CPU/memory profile can provide clues about resource-intensive operations even when the source code is unavailable. Sudden CPU spikes or increasing memory consumption may indicate repeated searches, nested loops, inefficient sorting, or excessive object creation. These are hypotheses that require controlled experiments to confirm.

1. Functional Decomposition: Divide the library module into book search, member validation, borrowing, returning, fine calculation, and record storage. Determine which operation coincides with each performance spike.

2. Execution Profile Analysis: Repeat the stress test with small, medium, and large datasets. If CPU usage increases sharply with dataset size, investigate repeated scans or nested processing. If memory usage continually increases, investigate possible memory leaks or unnecessary data retention.

3. Conceptual CFG: Model a search operation as Start → Validate Query → Search Records → Match Found? If yes, return the result; otherwise, continue searching until the records are exhausted, then return “Not Found.” Include loop exits, invalid queries, and error-handling paths.

4. Reconstructed Path Testing: Test empty libraries, one record, many records, successful searches, unsuccessful searches, and repeated queries. Use profiling to compare the execution time and memory consumption of each scenario. Where tools permit, monitor branches and function execution in the compiled program.

5. Stability Evaluation: Test realistic workloads and stress conditions, including repeated borrowing and returning operations. Check whether resource consumption stabilizes, whether requests finish within acceptable limits, and whether the module remains responsive under load.

Conclusion: Combining execution profiling with conceptual CFG analysis helps identify likely resource-intensive paths and guide targeted testing. Since an executable's profile does not directly reveal its source-level control flow, the reconstructed model should be treated as an evidence-based approximation rather than an exact representation.

Q6. Mobile Payment Gateway – Reverse Engineering State Transitions

The API responses for Pending, Success, Reversal, and Timeout can be used to reconstruct the gateway's state machine. The objective is to identify which events trigger each state change and prevent invalid transitions from corrupting payment records.

1. Functional Decomposition: Separate transaction initiation, payment processing, status reporting, timeout handling, and reversal processing.

2. Reconstructed State Model:

Pending
Success
Timeout

Reversal

Conceptual states; actual transitions depend on gateway rules.

3. Transition Testing: Test Pending → Success, Pending → Timeout, and eligible failed-payment → Reversal transitions. Verify the actual permitted transitions from API responses and documentation. An additional Success → Reversal transition should be tested only if the gateway supports post-success reversal.

4. Illegal Transition Testing: Attempt duplicate success callbacks, reversal of an unrelated transaction, repeated reversal requests, and conflicting status updates. Verify that the system rejects invalid changes and does not process the same payment twice.

5. Data Validation: Check transaction IDs, status codes, amounts, timestamps, and response consistency. Confirm that a timeout does not automatically imply that the payment failed unless the gateway's rules establish that outcome.

Conclusion: State Transition Testing is the appropriate method for reconstructing a payment workflow. Testing legal and illegal transitions, along with duplicate callbacks and timeout scenarios, helps maintain payment consistency and prevents incorrect transaction states.

Q7. Encrypted Communication Module – Fuzzing and Boundary Testing

An encrypted communication module exposes only inputs and encrypted outputs, so reverse engineering must focus on observable behavior. Carefully designed fuzzing can reveal validation failures, abnormal termination, or inconsistent error handling without requiring access to the source code.

1. Functional Decomposition: Divide the module into input parsing, data validation, encryption, output generation, and error handling.

2. Fuzzing Analysis: Supply malformed messages, missing parameters, unexpected data types, oversized inputs, invalid encodings, and unusual character sequences. Record whether the system rejects the input safely, produces a valid response, or crashes.

3. Boundary Value Analysis: Identify documented maximum message lengths and field sizes. Test inputs just below, exactly at, and just above each limit. Also test empty input, the smallest valid message, and malformed length fields.

4. Security-focused Test Cases: Test malformed structured data, unexpected delimiters, repeated special characters, and invalid encryption parameters. Verify that inputs cannot bypass validation or alter message interpretation. Use authorized test environments and controlled payloads.

5. Expected Results: Invalid inputs should produce controlled errors without crashes, memory corruption, or disclosure of sensitive information. Oversized data should be rejected or handled according to the specified protocol.

Conclusion: Fuzzing discovers unexpected input-handling failures, while BVA systematically tests known limits. Together they can reveal weaknesses associated with buffer overflow and injection risks, although secure implementation and specialized security analysis are also necessary to establish protection.

Q8. Insurance Premium Calculator – Statement Coverage Is Insufficient

A legacy insurance calculator achieves 100% statement coverage but still contains defects in multi-condition decisions. This indicates that the tests execute the statements without adequately examining how combinations of Boolean conditions affect premium calculations.

1. Functional Decomposition: Identify input validation, age classification, policy selection, risk assessment, discount calculation, and final premium generation.

2. Conceptual CFG: Consider the decision:

```
Eligible = (Age >= 18) AND (PolicyActive)
```

The true branch proceeds to premium calculation, while the false branch rejects the application. A test suite that executes both branches may still fail to isolate the individual effect of each condition.

3. Reconstructed Test Cases: Use three cases: age valid and policy active; age invalid and policy active; age valid and policy inactive. These cases demonstrate how the two conditions influence the decision. Extend the same principle to every compound decision in the actual calculator.

4. Coverage Strategy: Achieve branch coverage by testing every decision outcome. Identify independent paths using cyclomatic complexity and test each feasible path. For the full module, combine this with boundary tests for age, policy limits, and discount thresholds.

5. Expected Improvement: Compare calculated premiums against independently verified expected values. This catches both control-flow defects and incorrect arithmetic that code coverage alone cannot detect.

Conclusion: Statement coverage does not guarantee that compound conditions are adequately tested. Reconstructing the CFG and combining branch testing, feasible path testing, and calculation assertions reduces the risk of incorrect insurance premiums. Complete path coverage should only be claimed when all feasible paths have actually been examined.

Q9. Cloud File Synchronization – Lost File Defects

Lost-file reports and user activity logs can reveal weaknesses in how the synchronization tool handles file size, file type, network speed, and interrupted transfers. The likely issue may involve incomplete uploads, incorrect conflict resolution, or failure to resume a transfer.

1. Functional Decomposition: Divide the system into file scanning, metadata comparison, upload/download, conflict detection, retry handling, and synchronization confirmation.

2. Infer Equivalence Classes: For file size, identify empty, small, normal, maximum-supported, and oversized files. For file type, distinguish supported and unsupported formats. For network speed, examine stable, slow, interrupted, and unavailable connections.

3. Boundary Value Analysis: Test files immediately below, at, and above the maximum supported size. Include empty files and files near chunk-size boundaries if the implementation uses chunked transfers.

4. Stress and Recovery Testing: Synchronize many files simultaneously, interrupt the network during upload, resume the connection, and repeat synchronization. Check that partially transferred files are not marked complete and that conflicting versions are handled according to the system's policy.

5. Data Integrity Verification: Compare source and destination file counts, sizes, and cryptographic hashes. Confirm that a successful synchronization response means the destination contains the complete and correct file.

Conclusion: ECP helps reconstruct likely input categories, while BVA and stress testing expose size-related and concurrency-related failures. Recovery testing and integrity checks provide direct evidence that future synchronization operations do not silently lose data.

Q10. E-commerce Search and Filter – Reverse Engineering Decision Logic

SQL queries and their resulting datasets can reveal how the search module combines filters. By comparing query parameters with returned records, the tester can infer whether conditions are joined using AND, OR, or NOT and reconstruct the corresponding decision rules.

1. Functional Decomposition: Divide the module into keyword search, category filtering, price filtering, availability filtering, sorting, and pagination.

2. Identify Decision Logic: Suppose a query filters products by category and maximum price. If both conditions must be satisfied, the logic uses AND. If a product can match either of two categories, the logic may use OR. NOT can exclude discontinued or restricted products.

3. Decision Table:

Category matches	Price is within limit	Expected with AND
No	Yes	Exclude product
No	No	Exclude product
Yes	Yes	Include product
Yes	No	Exclude product

4. Reconstructed Test Cases: Test each row of the table, then examine combinations involving availability, multiple categories, excluded products, empty search terms, and missing filter values. Verify that sorting and pagination do not omit matching products.

5. Completeness Evaluation: Compare the actual SQL predicates and returned dataset with the expected rules. Include duplicate conditions, null values, and conflicting filters where supported.

If multiple filters interact, use decision tables to cover each meaningful rule without generating every possible combination unnecessarily.

Conclusion: Decision Table Testing makes complex filter behavior explicit and testable. Comparing expected results with actual query outputs helps identify incorrect AND/OR logic, missing exclusions, and incomplete search results, improving the reliability of the search module.